

Data Visualization Overview - CSE135

Summary

- **Dashboard Design:**
 - **Progressive disclosure:** “overview first, zoom and filter, then details on demand”
 - **5-second test:** can new users identify dashboard purpose and main finding in 5 seconds
 - **Show comparison:** indicate whether a metric is up, down, or flat relative to last period
 - If you cannot articulate what decision each chart supports, remove the chart.
 - **Annotation:** charts highlight the important insight of the data points
- **Types of Charts:**
 - **Pie chart:** few slices (2-3), emphasize one slice over all others, and exact comparison between slices isn't important
 - **Grouped bar chart:** compare individual metrics within the group
 - **Stacked bar chart:** emphasizes total volume of different metrics
 - **Line chart:** emphasizes trends over time (e.g. site traffic)
- **When to use tables:**
 - Small datasets (chart doesn't aid pattern recognition)
 - Columns have no shared quantitative axis
 - Exact-value lookup tasks
- **Table accessibility:**
 - <caption> for table identity, <th scope="col"> for header–cell association
 - “aria-sort” on sortable headers to announce current sort state
 - “aria-live="polite"” region to announce filter result counts
- **Use both:** Use charts to show trends and tables underneath them for confirmation and lookup. Table also provides more accessibility than charts (with Canvas). Can also use CSV download link instead of table.
- **Hybrid chart model:**
 - Server does heavy lifting, creates aggregated payload in a .JSON file. Browser gets the smaller .JSON file and uses [chart.js](#) to render
 - Render with Canvas/SVG in browser
 - Tools to look into: JS charting library, Grafana, Metabase
 - Cons: aggregated data must travel over the network, parsed by the browser, then rendered. Getting data to the renderer via sending .JSON files over the network and parsing it takes a long time

- **Problems with .JSON**

- Records repeat key names every time
- Parser is all or nothing. The browser sits idle during download, blocking the main thread during parsing
- Not streamable, cannot draw data as it arrives with progressive rendering
- Encoding text is less efficient than other formats, creating overhead
- **Alternative:** Apache Arrow (IPC) (lib = apache-arrow) is smaller size, faster speech, and streamable (record batches). “Zero-copy” approach eliminates need for parsing
- **Note:** if .JSON is a 5KB response or under, Apache Arrow is overkill and overengineered. Stick with .JSON that has native support

- **Data aggregation**

- **Server-side aggregation:** SQL command “GROUP BY” aggregates rows, reducing .JSON size. Loses individual record detail, but always best first move
 - Ex: Google Maps handle millions of geographic points — zoom out and you see a sample; zoom in and you fetch the detail for that viewport
- **Sampling:** send every Nth row or random sample. Good for pattern detection, not for exact counts
- **Progressive/on-demand loading:** Load summary first, fetch detail when user asks. Adds latency and requires API to support parameterized queries

- **Rendering technology:**

- **Canvas:** [chart.js](#) uses Canvas2D internally.
 - Uses immediate mode: draw and forget. issue a draw command, pixels are painted, and the shape goes to memory. To update: clear and redraw everything
 - **Pros:** fixed memory footprint regardless of shape count, pixel-level control with “getImageData()”, “toDataURL()” for native PNG export
 - **Cons:** No native interactivity, manual hit-testing required, <canvas> is invisible to screen readers, “devicePixelRatio” for crisp rendering on HiDPI displays
- **SVG:**
 - Uses retained mode: creates persistent DOM nodes.
 - **Pros:** Resolution independence, full CSS styling, native DOM events per element, accessibility, DevTools inspectability, real selectable text (instead of images)
 - **Cons:** Each element adds overhead for cascade, layout, and event dispatch. = DOM explosion. Max 50k elements

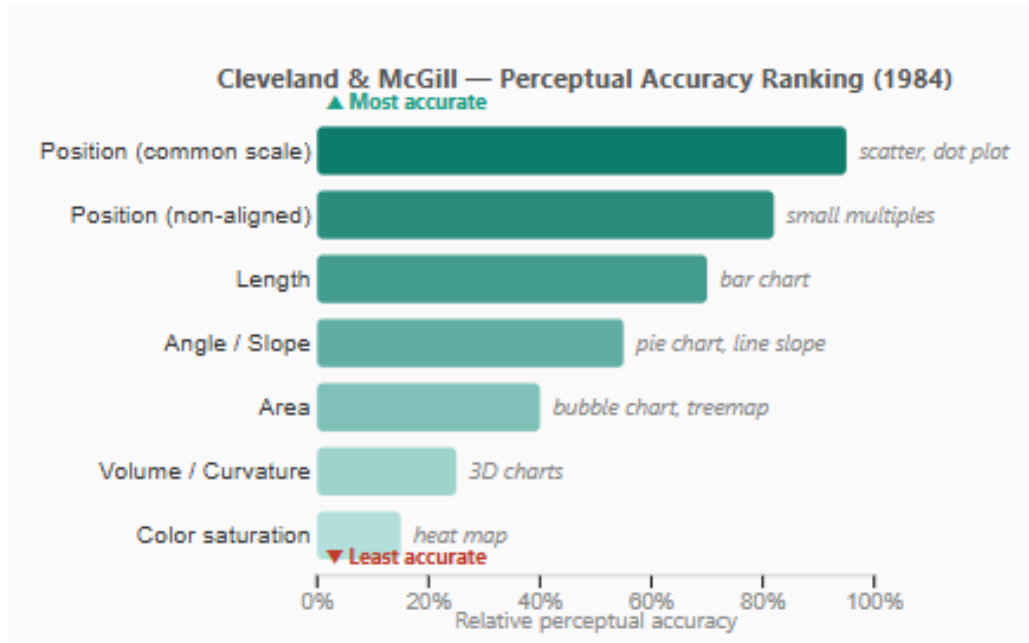
- [Chart.js](#)
 - Declarative/configuration-driven: you decide what you want (data, chart type, options), and the library handles rendering it for you
 - Requires less lines of code and less learning curve, but you get less control (compared to **D3**)

Images from CSE135 (for additional info):

Ways to visualize data:

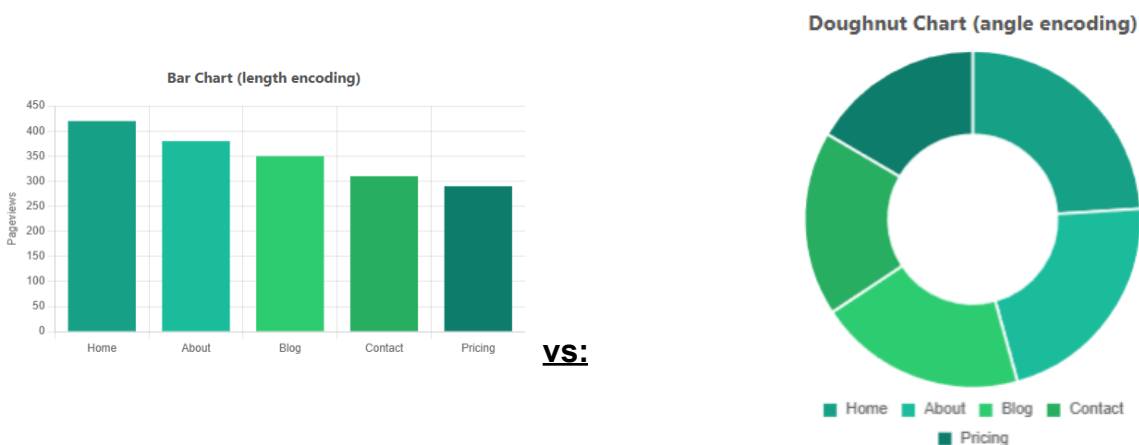
Visual Channel	Best For	Accuracy	Example Chart
Position (x, y)	Quantitative	Highest	Scatter plot, line chart
Length	Quantitative	Very high	Bar chart
Angle / Slope	Quantitative	Moderate	Pie chart, line slope
Area	Quantitative	Low	Bubble chart, treemap
Color hue	Categorical	N/A (categorical)	Colored scatter, legend groups
Color saturation / lightness	Quantitative (ordered)	Low	Choropleth map, heat map
Shape	Categorical	N/A (categorical)	Scatter with different point shapes

People's accuracy of differentiating values with different metrics:



(position = relative position of dots on graph. Length = relative height of bars on charts. Angle = relative size of sections in pie charts. Area = relative size of different shapes)

Example of ability to determine relative size:



(shows that relative size of angles is harder to decipher than bar height)

Chart Types (pros and cons)

Question	Chart Type	Best For	Watch Out For
How do categories compare?	Bar chart	Comparing values across discrete categories	Too many bars (>15) become unreadable
	Grouped bar	Comparing sub-categories within categories	More than 3–4 groups per cluster becomes confusing
	Lollipop	Same as bar but with less visual weight	Less familiar to some audiences showing the cultural nature of viz as they used to be common
What are the parts of a whole?	Pie / Donut	Showing a single dominant proportion	More than 4–5 slices becomes unreadable
	Stacked bar	Parts-of-whole across multiple categories	Interior segments hard to compare across bars
	Treemap	Hierarchical part-to-whole with many items	Small rectangles become illegible; area decoding is imprecise
How is data distributed?	Histogram	Distribution shape of a single variable	Bin width choice changes the story
	Box plot	Comparing distributions across groups	Hides multi-modality; requires statistical literacy
How does a value change over time?	Line chart	Continuous trends	Too many lines overlap; use small multiples instead
	Area chart	Volume / magnitude over time	Stacked areas distort upper layers
	Sparkline	Inline trend within text or table	No axes — shows shape, not value
What is the relationship between variables?	Scatter plot	Correlation, clusters, outliers	Overplotting with many points
	Bubble chart	Scatter + third variable as size	Area encoding is imprecise (see Section 3)
What is the current status?	Gauge	Single KPI against a target	Takes a lot of space for one number
	KPI card	Single number with trend arrow or sparkline	No context without comparison period